



Object-Oriented and Classical Software Engineering

Eighth Edition, WCB/McGraw-Hill, 2011



UNIVERSITY OF THE FREE STATE • UNIVERSITEIT VAN DIE VRYSTAAT • YUNIVESITHI YA FREISTATA

P O Box 339, Internal Box 65, Bloemfontein, 9300 • Republic of South Africa

Tel: +27 (0)51-401 2745 • Fax: +27 (0)51-401 2754

CHAPTER 2

SOFTWARE LIFE-CYCLE MODELS

2.1 Software Development in Theory

- Ideally, software is developed as described in Chapter 1
 - Linear
 - Starting from scratch

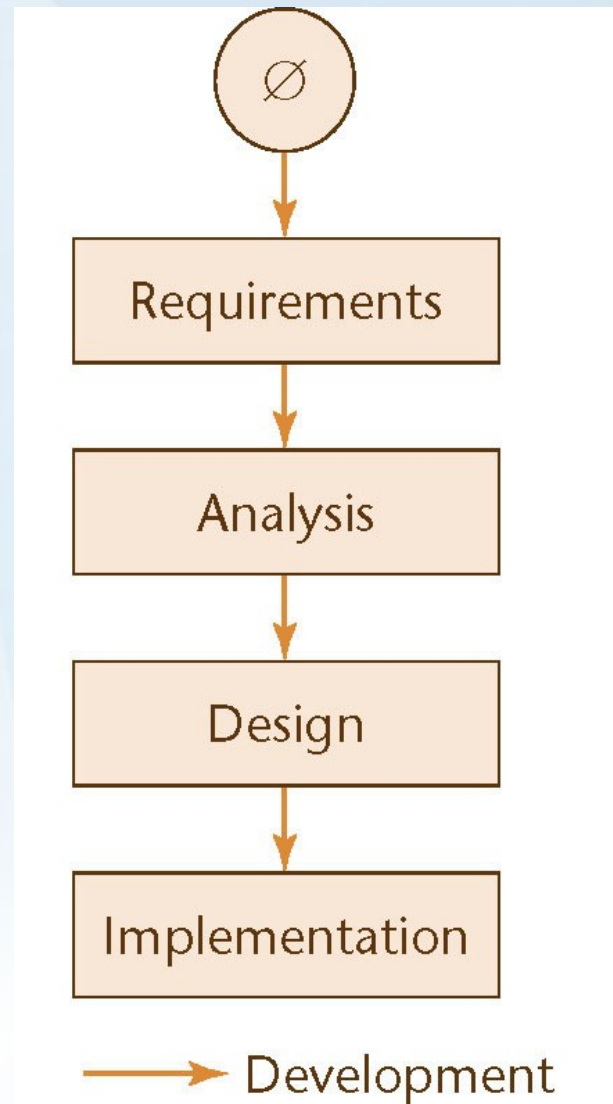


Figure 2.1

Software Development in Theory

- However, software development in practice is different:
 - In the real world, software development is totally different:
 - We make mistakes
 - The client's requirements change while the software product is being developed

2.2 Winburg Mini Case Study

- **Episode 1:** The first version is implemented
- **Episode 2:** A fault is found
 - The product is too slow because of an implementation fault
 - Changes to the implementation are begun
- **Episode 3:** A new design is adopted
 - A faster algorithm is used
- **Episode 4:** The requirements change
 - Accuracy has to be increased
- **Epilogue:** A few years later, these problems recur

Evolution-Tree Model

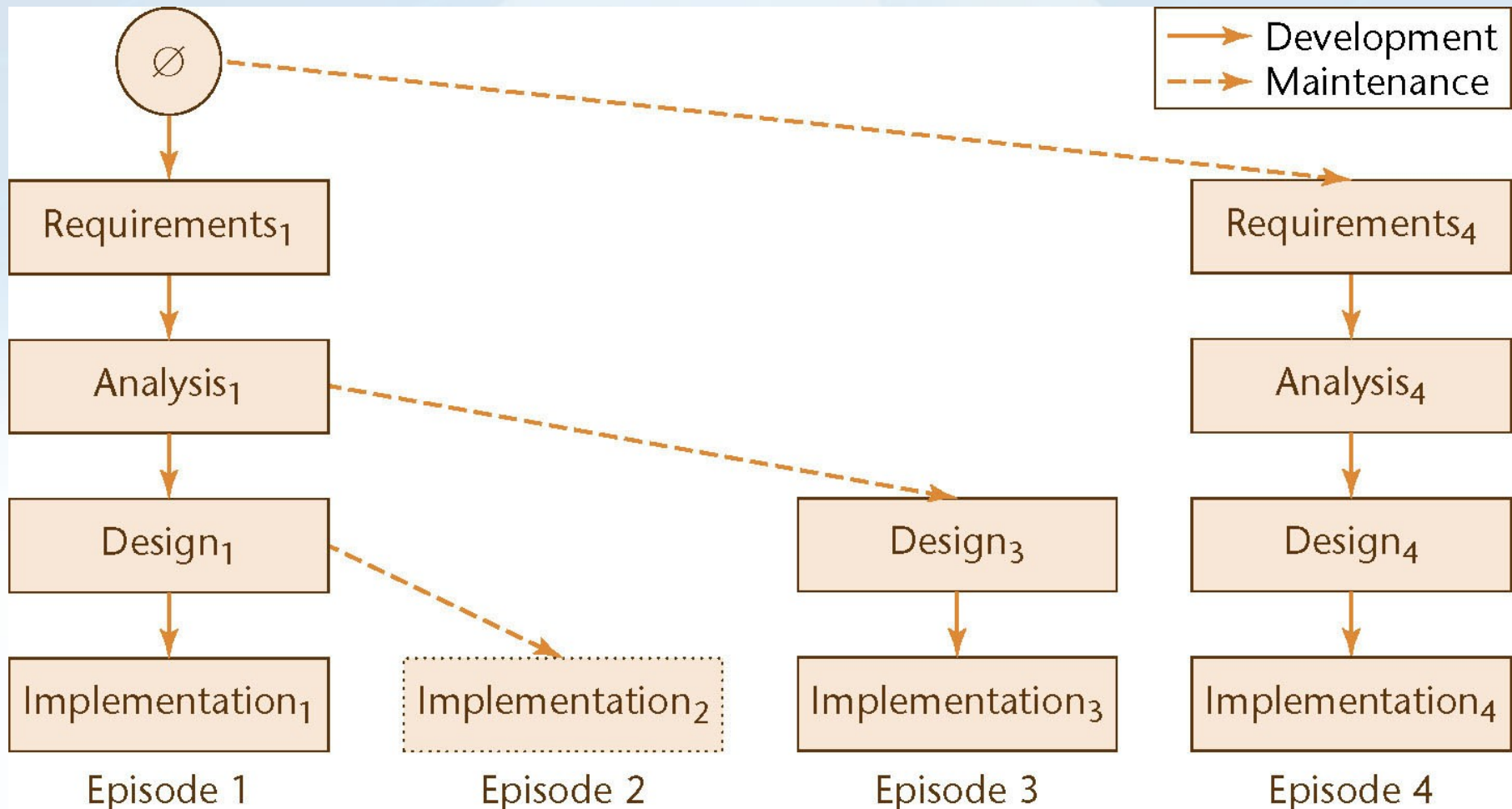


Figure 2.2

Waterfall Model

- The linear life cycle model with feedback loops
 - The waterfall model cannot show the order of events

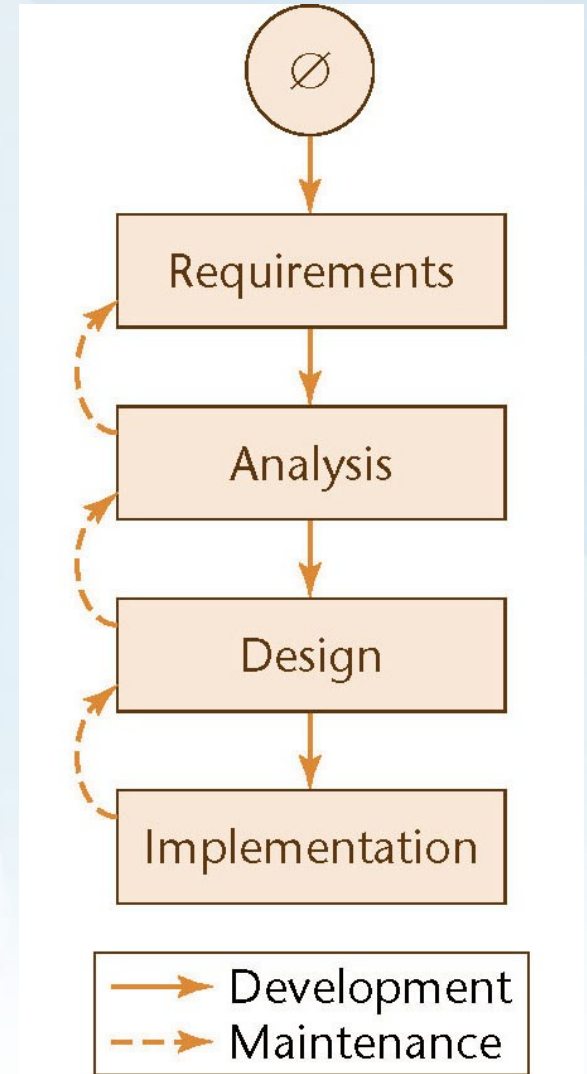


Figure 2.3

2.3 Lessons of the Winburg Mini Case Study

- Software development in the real world is chaotic and many changes are needed to the software product.
- Why?
 - Software professionals are human and make mistakes.
 - Real world is continually changing. Therefore the software product must also change to keep up and be relevant.

2.5 Iteration and Incrementation

- In real life, we cannot speak about “the analysis phase”
 - Instead, the operations of the analysis phase are spread out over the life cycle
- The basic software development process is iterative
 - Each successive version is intended to be closer to its target than its predecessor

Miller's Law

- At any one time, we can concentrate on only approximately seven *chunks* (units of information)
- To handle larger amounts of information, use *stepwise refinement*
 - Concentrate on the aspects that are currently the most important
 - Postpone aspects that are currently less critical
 - Every aspect is eventually handled, but in order of current importance
- This is an *incremental* process

Classical Phases vs Workflows

- Sequential phases do not exist in the real world
- Instead, the five core workflows (activities) are performed over the entire life cycle
 - Requirements workflow
 - Analysis workflow
 - Design workflow
 - Implementation workflow
 - Test workflow

Workflows

- All five core workflows are performed over the entire life cycle
- However, at most times one workflow predominates
- Examples:
 - At the beginning of the life cycle
 - The requirements workflow predominates
 - At the end of the life cycle
 - The implementation and test workflows predominate
- Planning and documentation activities are performed throughout the life cycle

Iteration and Incrementation

- Iteration is performed during each incrementation

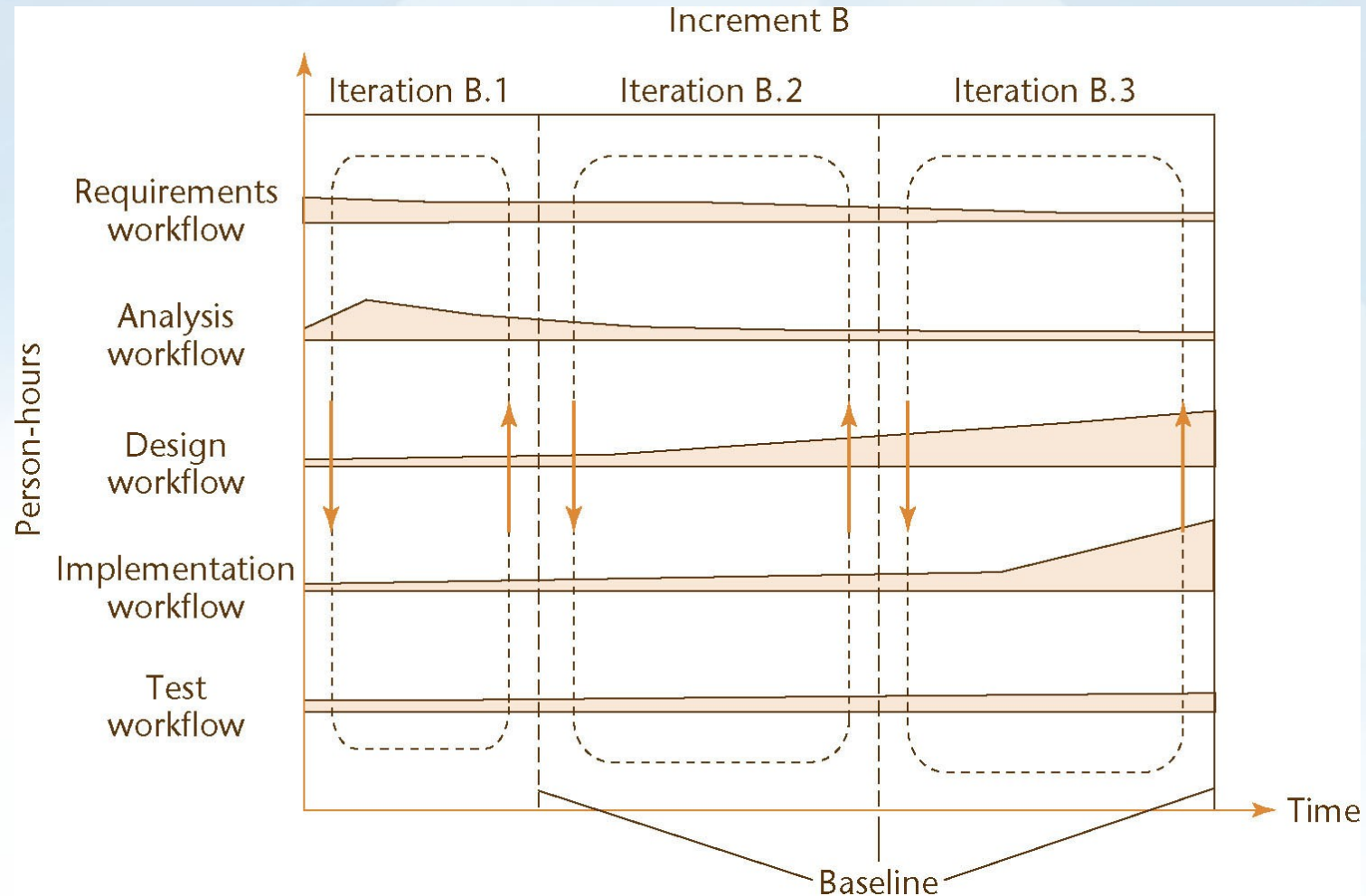


Figure 2.5

Iteration and Incrementation (contd)

- Again, the number of iterations will vary—it is not always three

2.7 Risks and Other Aspects of Iter. and Increm.

- We can consider the project as a set of mini projects (increments)
- Each mini project extends the
 - Requirements artifacts
 - Analysis artifacts
 - Design artifacts
 - Implementation artifacts
 - Testing artifacts
- The final set of artifacts is the complete product

Risks and Other Aspects of Iter. and Increm. (contd)

- During each mini project we
 - Extend the artifacts (incrementation);
 - Check the artifacts (test workflow); and
 - If necessary, change the relevant artifacts (iteration)

Risks and Other Aspects of Iter. and Increm. (contd)

- Each iteration can be viewed as a small but complete waterfall life-cycle model
- During each iteration we select a portion of the software product
- On that portion we perform the
 - Classical requirements phase
 - Classical analysis phase
 - Classical design phase
 - Classical implementation phase

Strengths of the Iterative-and-Incremental Model (contd)

- We can *mitigate* (resolve) risks early
 - Risks are invariably involved in software development and maintenance
- We have a working version of the software product from the start
 - The client and users can experiment with this version to determine what changes are needed
 - Variation: Deliver partial versions to smooth the introduction of the new product in the client organization
- There is empirical evidence that the iterative-and-incremental life-cycle works.

2.9 Other Life-Cycle Models

- The following life-cycle models are presented and compared:
 - Code-and-fix life-cycle model
 - Waterfall life-cycle model
 - Rapid prototyping life-cycle model
 - Open-source life-cycle model
 - Agile processes
 - Synchronize-and-stabilize life-cycle model
 - Spiral life-cycle model

2.9.1 Code-and-Fix Model

- No requirements
- No specifications
 - Maintenance nightmare
- No design

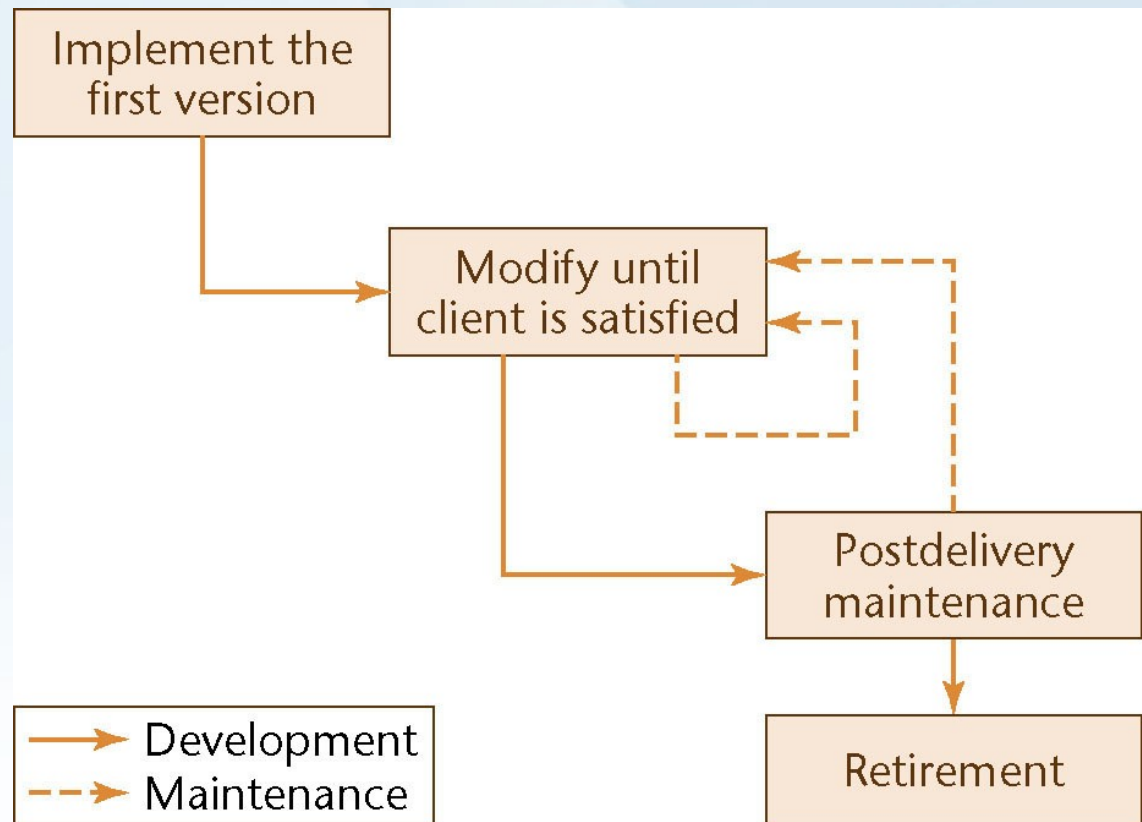


Figure 2.8

Code-and-Fix Model (contd)

- Develop system and rework until client is satisfied
- Sufficient for 100-200 LOC
- Chances of a regression fault are larger
- The easiest way to develop software
- The most expensive way
- Used often
 - Especially where performance is measured in terms of LOC

2.9.2 Waterfall Model

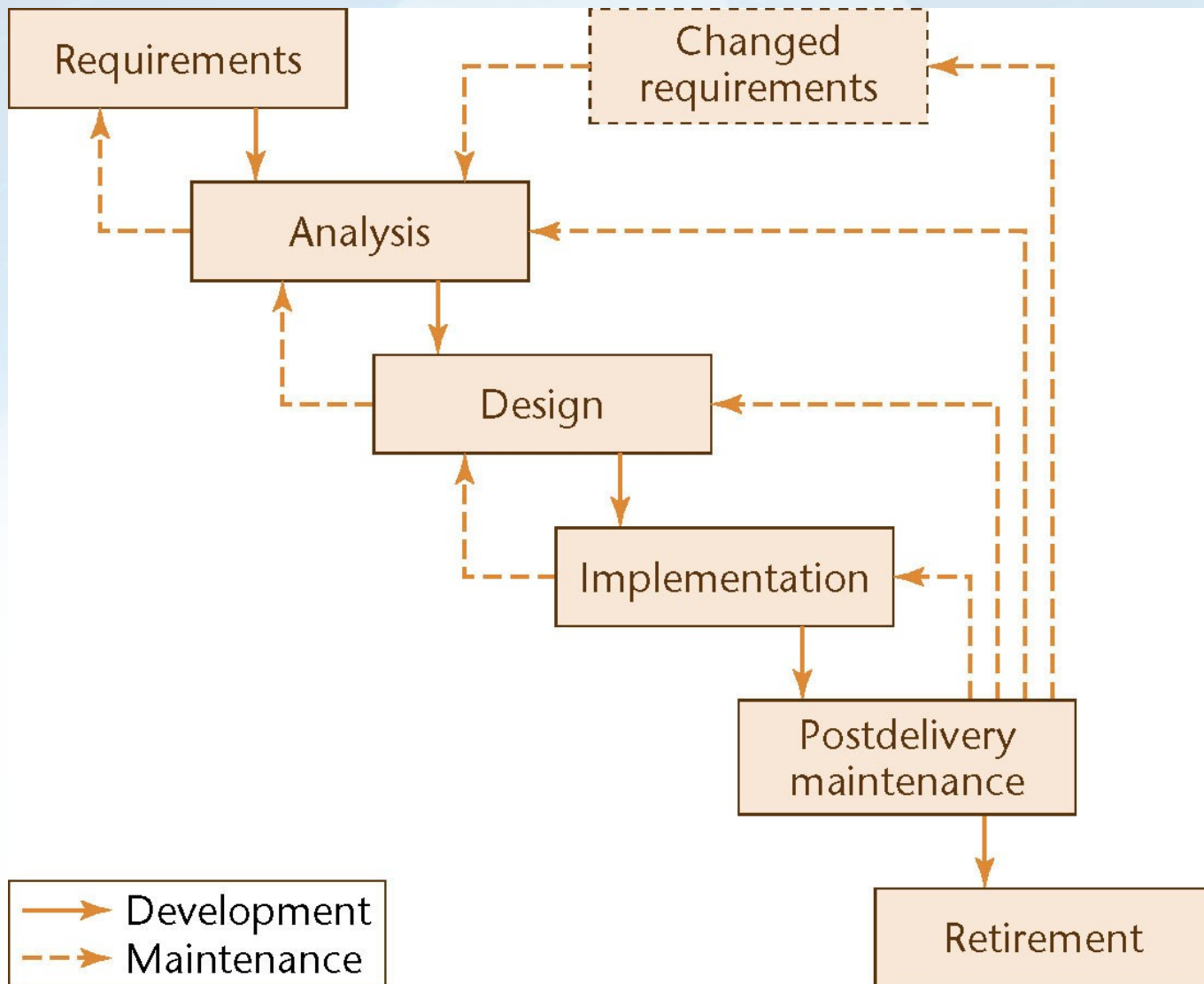


Figure 2.9

2.9.2 Waterfall Model

- Characterized by
 - Feedback loops
 - During development and
 - After delivery
 - Documentation-driven
 - No phase is complete until the documentation is finalised through approval by the Software Quality Assurance (SQA) group
 - Same applies for modifications – changes in products of earlier phase due to a feedback loop – then the documents are again reviewed by SQA group.
- Testing is present in every phase and *throughout each phase*
 - Regression testing occurs during maintenance as well

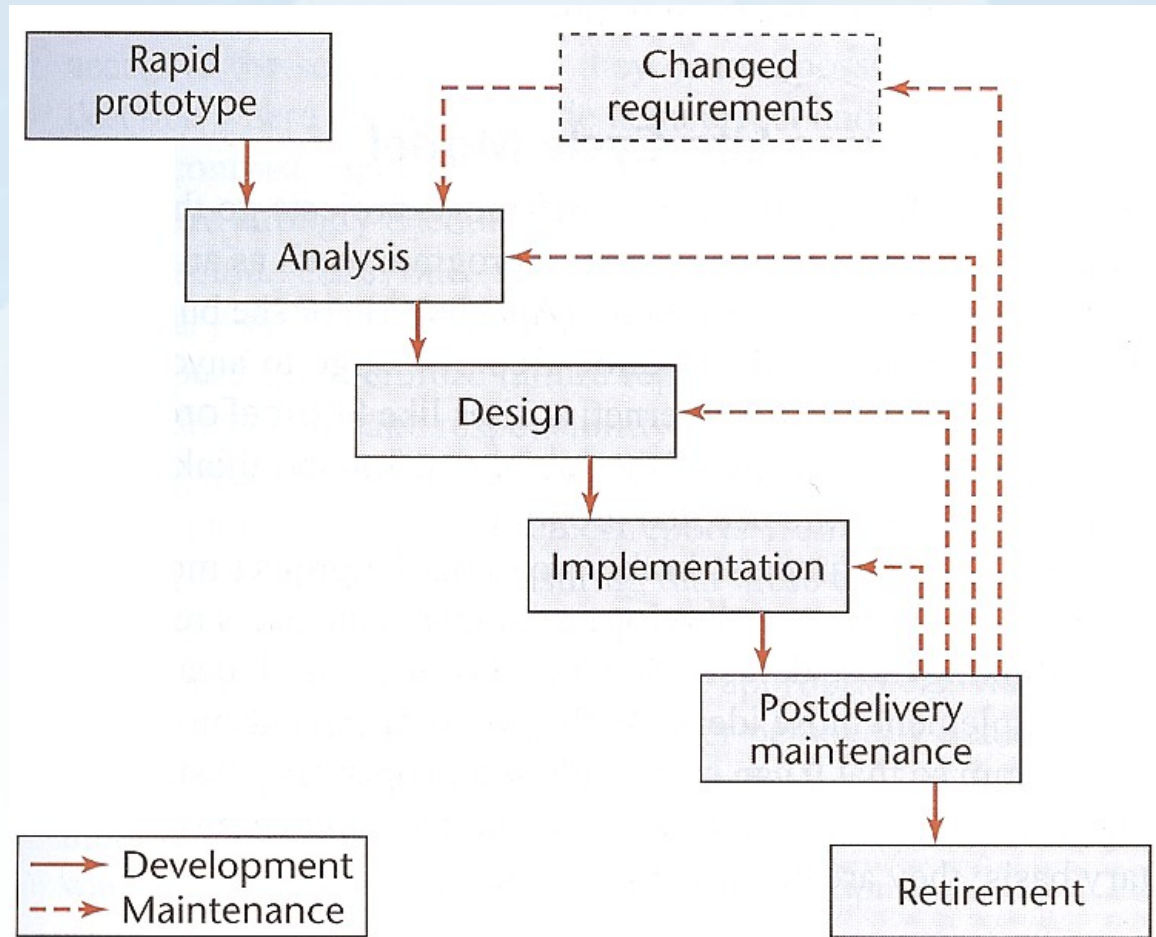
Software Quality Assurance

- **Software quality assurance (SQA)**
 - Means of monitoring the software engineering processes and methods used to ensure quality.
 - Done by means of audits of the quality management system under which the software system is created.
 - Audits are backed by one or more standards
- Software quality control includes reviewing requirements documents, and software testing.
- SQA encompasses the entire software development process, which includes processes such as software design, coding, source code control, code reviews, release management.

2.9.3 Rapid-Prototyping Model

- Rapid prototype
 - Working model that is functionally equivalent to *a subset* of the product.
 - For example
 - May show data capture and report printing but have no file updating or database connectivity or error handling

Rapid-Prototyping Model (contd)



Rapid-Prototyping Model (contd)

- First step in life-cycle
 - Build a rapid prototype
 - Allow client to interact with rapid prototype
- Specification document is drawn up once the client is satisfied that the prototype represents what they want
- Assurance that the product meets the clients needs

Rapid-Prototyping Model (contd)

- Advantages
 - Essentially a linear process since the feedback loops, although present, are unlikely to be needed
 - Specification document is compiled using the rapid prototype
 - Validation by client means specification document should be correct
 - Design team gains insight from rapid prototype
 - Worst case scenario – How not to do it

Rapid-Prototyping Model (contd)

- Implementation phase is next
 - Design of prototype usually means less need for repair during this phase
- Client accepts product and it is installed
- Postdelivery maintenance
 - Life-cycle is reentered at the requirements, analysis, design or implementation phase depending on the type of maintenance required

2.9.4 Open-Source Life-Cycle Model

- Two informal phases
- First, one individual builds an initial version
 - Made available via the Internet
- Then, if there is sufficient interest in the project
 - The initial version is widely downloaded
 - Users become co-developers
 - The product is extended
- Key point: Individuals generally work voluntarily on an open-source project in their spare time

The Activities of the Second Informal Phase

- Reporting and correcting defects
 - Corrective maintenance
- Adding additional functionality
 - Perfective maintenance
- Porting the program to a new environment
 - Adaptive maintenance
- The second informal phase consists *solely* of postdelivery maintenance
 - The word “co-developers” on the previous slide should rather be “co-maintainers”

Open-Source Life-Cycle Model (contd)

- Postdelivery maintenance life-cycle model

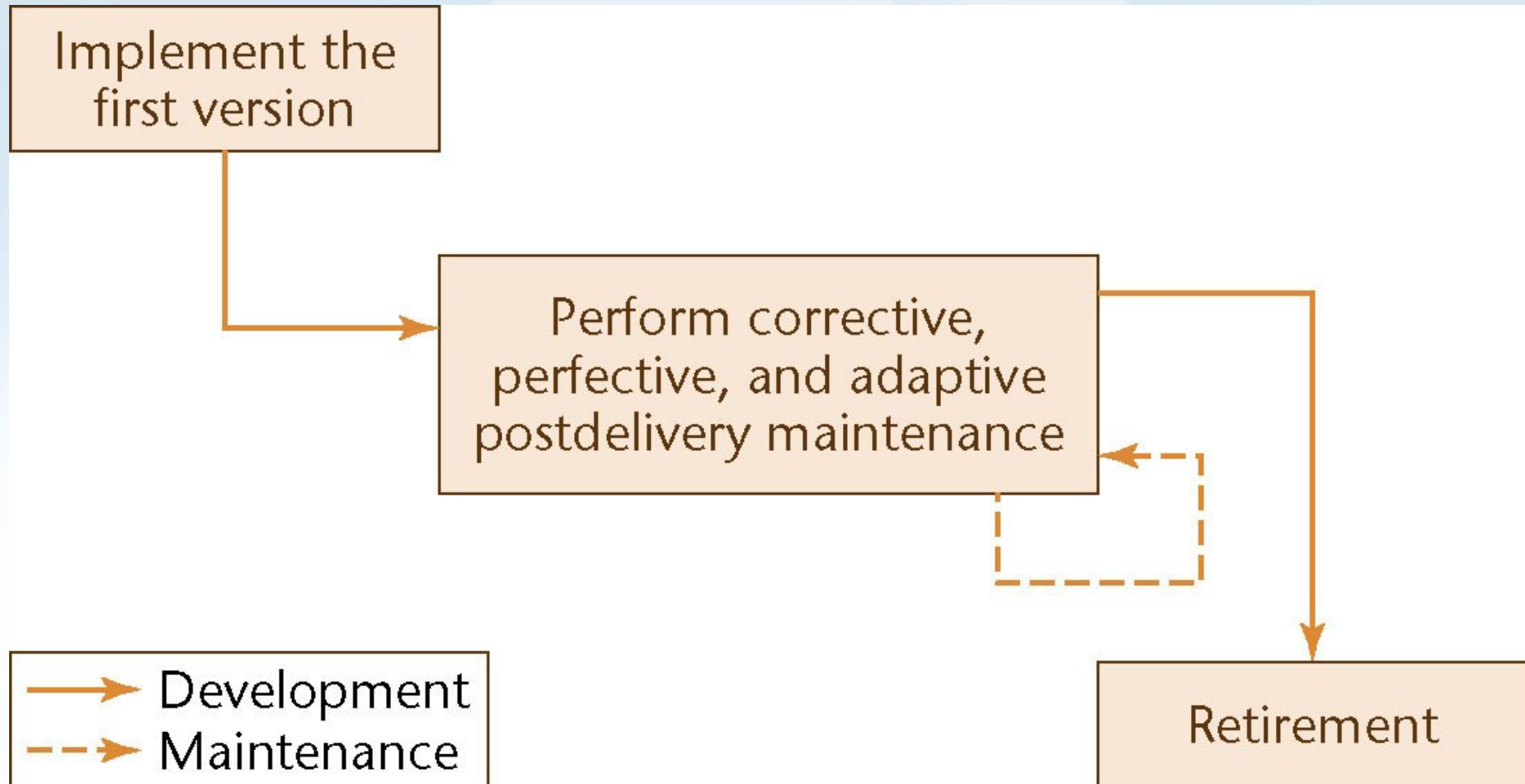


Figure 2.11

Open-Source Life-Cycle Model (contd)

- Closed-source software is maintained and tested by employees
 - Users can submit failure reports but never fault reports (the source code is not available)
- Open-source software is generally maintained by unpaid volunteers
 - Users are strongly encouraged to submit defect reports, both failure reports and fault reports

2.9.5 Agile Processes

- Extreme programming
 - Based on the iterative-and-incremental model
- First step
 - Development team determines features wanted by client
 - For each feature client is informed of the length of time required and the cost
 - Corresponds to requirements and analysis workflows of iterative-and-incremental model

Agile Processes (contd)

- Client performs cost-benefit analysis
 - Based on time and cost estimates of development team and estimated benefits to the company
- Selects which features he wants included
- Product is broken down into tasks
- Test cases are drawn up for tasks (test-driven development)

Agile Processes (contd)

- Programmers work in pairs on a single computer
- Alternate coding every 15-20 minutes
- When not typing, the programmer checks the code of his partner
- Task is then integrated into the product build
- Tasks are implemented in parallel by a number of team pairs
- Integration is continuous
- Team members change partners every day

Unusual Features of XP

- The computers are put in the center of a large room lined with cubicles
- A client representative is always present
- Software professionals cannot work overtime for 2 successive weeks
- No specialization
 - All team members work on requirements, analysis, design and implementation
- Design is modified while the product is being built

Agile Processes (contd)

- Common feature is a daily meeting
 - Compulsory for all team members
 - Members must stand
 - To ensure the meeting lasts no more than 15 minutes
 - Each member must answer
 - What have I done since yesterday's meeting?
 - What am I working on today?
 - What problems are preventing me from achieving this?
 - What have we forgotten?
 - What did I learn that I would like to share with the team?

Agile Processes (contd)

- The aim of a stand-up meeting is
 - To raise problems
 - Not solve them
- Solutions are found at follow-up meetings, preferably held directly after the stand-up meeting

Agile Processes (contd)

- In conclusion
 - Agile processes appear to be a useful approach to building small-scale software products when the client's requirements are vague
 - Also, some of the proven features of agile processes can be effectively utilized within the context of other life-cycle models

2.9.6 Synchronize-and- Stabilize Model

- Microsoft's life-cycle model
- Requirements analysis — interview potential customers
- Draw up specifications
- Divide project into 3 or 4 builds
- Each build is carried out by small teams working in parallel

Synchronize-and Stabilize Model

- At the end of the day — *synchronize* (test and debug)
- At the end of the build — *stabilize* (freeze the build)
- Components always work together
 - Get early insights into the operation of the product

2.9.7 Spiral Model

- Simplified form
 - Waterfall model plus risk analysis preceding each phase

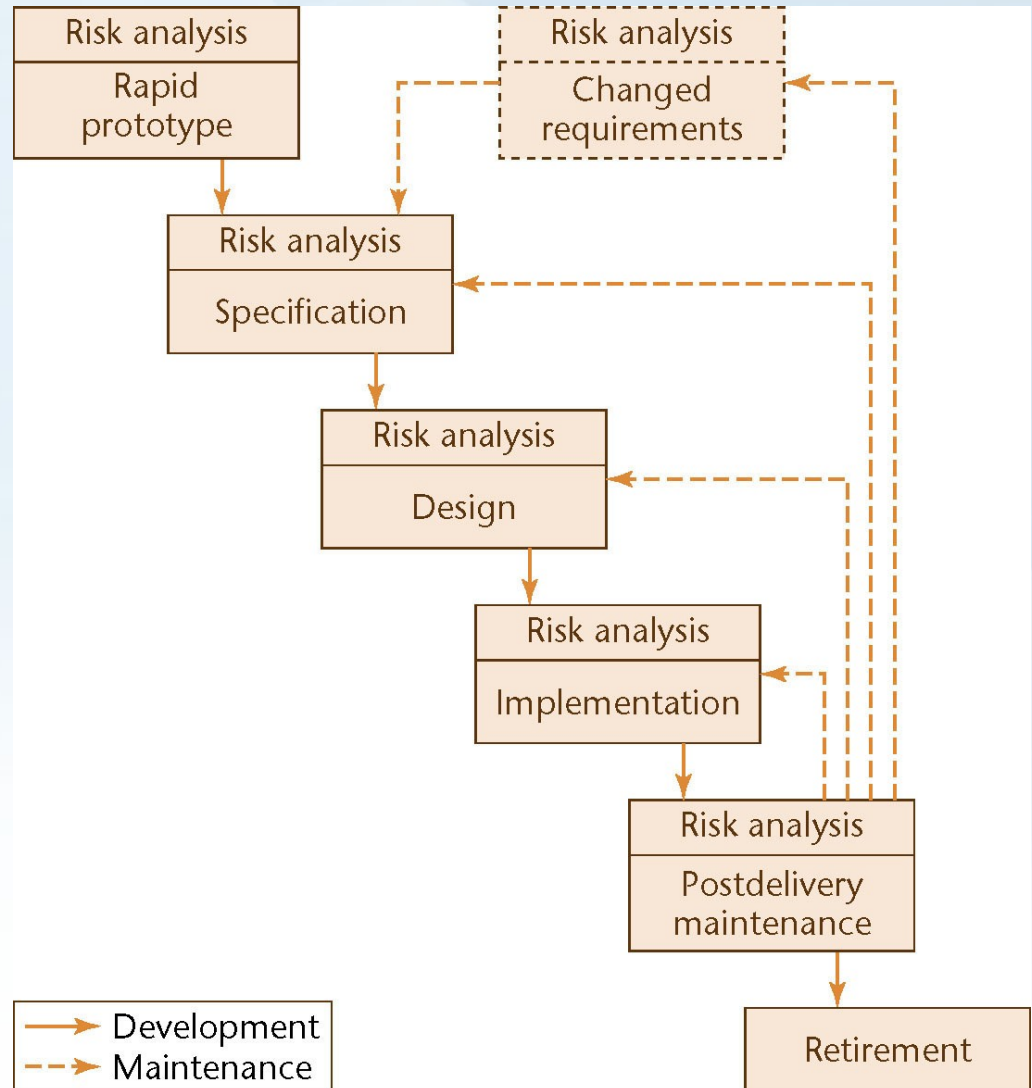


Figure 2.12

Spiral Model

- Key point
 - If all risks cannot be mitigated, the project is immediately terminated

Full Spiral Model

- Precede each phase by
 - Alternatives
 - Risk analysis
- Follow each phase by
 - Evaluation
 - Planning of the next phase